

Context-Aware Verification Cost Estimation for Speculative LLM Serving

Revised proposal · CS 6222 · with preliminary results

Problem

vLLM's DSpark speculative decoding chooses how many draft tokens to verify each step by maximising expected accepted tokens per unit of predicted step time. The prediction comes from a cost table profiled once at startup: the drafter is priced by request count and the verifier by total token count. Every profiling run assumes a single fixed context length (VLLM_ADAPTIVE_VERIFICATION_PROFILE_CONTEXT_LEN, default 8192), so the live batch's actual context never enters the estimate. Under continuous batching, where batch composition changes every iteration, two batches with identical token counts but different contexts are priced identically. vLLM's maintainers identify sequence-length-dependent cost estimation as an open problem (issue #52057).

Preliminary results

We have instrumented vLLM 0.28.0 and measured this directly, on an RTX 4060 laptop and on an NVIDIA H100 (PACE ice-gpu, FlashAttention 3, 567k-token KV cache with no block aliasing).

Experiment	Question	Result
E1	Does context change step cost at fixed token count?	Up to 3.7× spread; stock table over-predicts short-context batches by 73%
E2	Does cost depend on <i>which</i> request receives extra tokens?	0.21% — smaller than a duplicate-cell drift control. No.
E3	Does stock DSpark run, and does the manager engage?	Yes. 47.9% acceptance, 1.44 accepted tokens per draft
E5	The same question inside the CUDA-graph regime	Up to 9.6× spread
E6	Both sweeps on production hardware (H100, FA3)	7.8× with graphs; ~1% without

Two results structure the rest of the work. First, the cost surface decomposes as a per-step floor plus KV-streaming time, $k \cdot \Sigma(\text{seq_lens})$, where k is the time to read one token of KV cache. On the H100, k is constant to $\pm 1\%$ at $0.0356 \mu\text{s}$ per token across a $16\times$ range of batch sizes, implying 3,221 GB/s — 96% of that GPU's specified memory bandwidth. A model fitted this way predicted a later experiment to within 0.4% on batch shapes it was not fitted on.

Second, the error is **gated on CUDA graphs**. With graphs active the per-step floor is ~ 2 ms and KV reads dominate; in eager mode the floor is ~ 8 ms of kernel-launch overhead, which absorbs the KV reads entirely. Production serving uses CUDA graphs, so the regime where the cost model is blindest is the regime that matters.

Hypothesis

Verification step cost is governed by a per-step floor and by KV streaming time, and the KV term is observable only once it exceeds the floor:

$$\text{cost} \approx \max(\text{floor}(\text{num_reqs}, \text{num_tokens}, \text{exec_mode}), k \cdot \Sigma(\text{seq_lens}))$$

This makes falsifiable predictions. The most direct: because vLLM profiles at 8192 tokens of context, every error measured so far has been over-prediction; above 8192 the sign must flip to under-prediction, which is the more dangerous failure because it causes the scheduler to over-commit verification and miss latency targets.

Proposed work

Following feedback that the original scope was too broad, we have dropped the cost-aware per-request allocation policy (E2 shows it has no cost signal to exploit) and the online estimator design study. The remaining work is:

- **Characterise the error.** Complete the measurement across the profiling boundary and across model sizes, and replicate issue #52057's predicted-versus-actual figure using vLLM's own profiled table against real generation steps rather than synthetic batches.
- **Establish the ceiling first.** Estimate the best-possible speedup by replaying the budget decision offline against measured true step times. If perfect prediction buys only a few percent, we report a measurement study rather than implementing a scheduler.
- **Select the functional form.** Fit additive, max-like and blended models on held-out batch shapes, separately for the graphed and eager regimes, and check that the recovered *k* matches the value derived independently from bandwidth.
- **Implement one improvement.** A context-conditioned correction to the verify-cost lookup, with *k* estimated online, leaving the existing CUDA-graph step function intact.
- **Evaluate.** Throughput, TPOT and p95/p99 latency against stock DSpark and against fixed-length verification, under workloads varying concurrency and context heterogeneity.

Evaluation and success criteria

Primary metric is predicted-versus-actual verification-time error (bias, MAE, RMSE, *r*), reported separately per execution regime. Secondary metrics are output-token throughput, TPOT and tail latency. The project succeeds if it either (a) demonstrates a measurable serving improvement from a context-conditioned cost model, or (b) establishes, with a credible oracle bound, that the mispricing does not materially affect serving performance and characterises when it would.

Risks

Risk	Mitigation
Adaptive verification requires FlashAttention 3, available only on Hopper (compute capability 9.x). A100, L40S and V100 cannot run it.	Confirmed working on PACE ice-gpu H100/H200 nodes; all production-representative measurements are taken there.
The oracle bound may show little headroom.	Run it before implementing. A negative result with a measured ceiling is a legitimate outcome and is reported as such.
Results so far use a single model per platform.	Repeat the key sweeps on a larger model; <i>k</i> scales with KV bytes per token, which is a checkable prediction.

All experiments, raw measurements and analysis are version-controlled at github.com/adixg/vllm-specdecode-cost-model. A bug found while setting up this work — Qwen3.5-family DSpark drafts being misrouted to the DeepSeek-V4 draft model — has been submitted upstream as [vllm-project/vllm#56641](https://github.com/vllm-project/vllm/issues/56641).